
Reproducible Training-Free Inference Acceleration on Pythia-70M

Ruoyu Fu
NLP Course Final Project
Code: GitHub repository

Abstract

Efficient inference is a practical bottleneck for autoregressive language models because attention computation and the KV cache grow with context length. This work studies training-free inference modifications on Pythia-70M: a reduced-KV grouped-query attention (GQA) conversion and two KV cache compression strategies implemented with KVPress, Knorm and StreamingLLM. We evaluate chunked teacher-forced perplexity, cached next-token perplexity, time to first token (TTFT), time per output token (TPOT), and throughput on WikiText-103 and a PG-19 sample. KVPress improves throughput in several CPU settings, especially WikiText at 128/512-token contexts, while the training-free GQA conversion stores only 2 KV heads but severely degrades perplexity and does not provide a usable speed-quality tradeoff. The experiments can be reproduced with the scripts included in the repository.

1 Introduction

Autoregressive decoding repeatedly evaluates a transformer while maintaining a key-value (KV) cache for previous tokens. This cache avoids recomputing attention states, but its memory and attention cost scale with sequence length. Recent efficient-inference work therefore attacks the problem from several directions: using fewer KV heads (Ainslie et al., 2023), more IO-aware attention kernels (Dao et al., 2022), or compressing the KV cache during generation (Xiao et al., 2023; NVIDIA, 2024).

This project focuses on methods that can be reproduced without training a new model. We use Pythia-70M (Biderman et al., 2023), a small GPT-NeoX model, as the common backbone. The assignment asks for both quality and speed evaluation on datasets such as WikiText and PG-19, so we report two complementary quality metrics. First, chunked perplexity is computed with teacher forcing on fixed 512-token chunks and is not affected by KVPress, because KVPress acts on generation caches rather than full-context forward passes. Second, cached perplexity evaluates next-token prediction after a prefill pass whose KV cache may be compressed. This second metric is the appropriate quality signal for KVPress in this implementation.

The main contribution is a reproducible comparison under a CPU-only course environment. The results show that reduced-KV GQA is not a reliable drop-in conversion for a model trained with standard multi-head attention, while KV cache compression can help on some contexts but is sensitive to runtime overhead.

2 Methods

Baseline. The baseline loads the local `EleutherAI/pythia-70m` checkpoint through Hugging Face Transformers and uses the default scaled dot-product attention (SDPA) implementation. No weights are changed.

Reduced-KV GQA conversion. Pythia-70M uses 8 attention heads. In the GQA variant, we compute the original query, key, and value states, average the key/value heads into 2 grouped KV heads, and run custom grouped attention without materializing repeated K/V states. The dynamic cache therefore stores tensors with shape `[batch, 2, seq, head_dim]` rather than 8 KV heads. This is still a training-free conversion rather than the fine-tuned GQA recipe of Ainslie et al. (2023), so quality degradation is expected.

KVPress cache compression. We evaluate two KVPress modes at compression ratio 0.5. Knorm keeps tokens with larger key norm scores. StreamingLLM keeps attention-sink tokens and recent tokens, following the observation that early sink tokens are important for stable long-context decoding (Xiao et al., 2023). In both cases, compression is applied after the prefill stage via forward hooks on GPT-NeoX attention layers. The implementation records per-layer cache lengths before and after compression, e.g., 512 cached positions are compressed to 256.

Metrics. All methods use the same token windows. Chunked PPL uses the first 4096 tokens and resets context every 512 tokens: each chunk input predicts the next token inside that chunk, with no KV state carried across chunk boundaries. Cached PPL uses tokens 0–511 as prefill context and scores tokens 512–767 one by one with ground-truth continuation tokens fed back into the cache; KVPress compresses the prefill cache before this scoring step, while baseline and GQA keep their normal caches. KV events count layer-level KVPress compression calls from cached-PPL scoring plus warm-up and measured speed prefills. Speed uses the first 128, 512, or 1024 tokens as prompt and manually decodes 64 greedy tokens. Each setting has one warm-up and three measured runs. TTFT is wall time from prefill start to selecting the first token; TPOT is decode time divided by the remaining generated tokens; throughput is generated tokens per end-to-end second.

3 Experimental setup

We evaluate the first 4096 tokens from the test split of WikiText-103 (Merity et al., 2016) and from one PG-19 test sample (Rae et al., 2020), matching the course requirement that PG-19 can be evaluated with a single long sample. The local environment is CPU-only with PyTorch 2.6.0, Transformers 4.56.2, Datasets 3.6.0, and KVPress 0.5.3. Since CUDA is unavailable, FlashAttention is not used as a main experimental condition.

The exact reproduction command is:

```
cd finalproj
.\scripts\run_matrix.ps1
```

This script runs baseline, GQA, KVPress-Knorm, and KVPress-StreamingLLM on both datasets and regenerates `comparison_quality.*` and `comparison_speed.*`.

4 Results and discussion

Table 1 summarizes quality. The chunked-PPL column should be used to compare baseline and GQA under the same fixed chunking. The cached-PPL column should be used to compare KVPress cache-compression behavior, because it scores the same continuation tokens after prefill. GQA causes a large degradation: WikiText chunked PPL rises from 63.72 to 2289.96, and PG-19 chunked PPL rises from 33.57 to 797.49. This confirms that converting a pretrained MHA checkpoint into 2 cached KV heads by simple averaging is not quality-preserving.

The WikiText baseline has much lower cached PPL than chunked PPL. To check whether this was a cache-scoring artifact, we recomputed ordinary teacher-forced PPL on the identical continuation slice: the first 512 tokens were used as context and the next 256 tokens were scored. The same-slice teacher-forced PPL was 13.088, while cached autoregressive PPL was 13.089. This small difference indicates that cached scoring matches standard teacher forcing on the same tokens. The gap from the 63.72 chunked PPL is therefore caused by the measurement protocol: chunked PPL averages over 4095 tokens and resets context every 512 tokens, whereas cached PPL uses one fixed local continuation with an intact 512-token prefill. We therefore compare cached PPL only across methods on the same cached-PPL slice, not against chunked PPL as an absolute model-quality number.

Table 1: Quality metrics. Chunked PPL resets context every 512 tokens over the first 4096 tokens; cached PPL scores tokens 512–767 after a 512-token prefill cache.

Dataset	Method	Chunked PPL	Cached PPL	KV events	Avg. comp.
WikiText	Baseline	63.72	13.09	0	–
WikiText	GQA-2KV	2289.96	3046.28	0	–
WikiText	KVPress-Knorm	63.72	52.44	78	0.50
WikiText	KVPress-StreamingLLM	63.72	61.30	78	0.50
PG-19	Baseline	33.57	29.60	0	–
PG-19	GQA-2KV	797.49	781.27	0	–
PG-19	KVPress-Knorm	33.57	31.76	78	0.50
PG-19	KVPress-StreamingLLM	33.57	29.49	78	0.50

Table 2: Throughput change relative to baseline. Positive values indicate faster generation.

Dataset	Method	128 ctx	512 ctx	1024 ctx
WikiText	GQA-2KV	-30.7%	+23.1%	-42.7%
WikiText	KVPress-Knorm	+31.3%	+49.7%	-14.3%
WikiText	KVPress-StreamingLLM	+35.3%	+50.3%	-17.5%
PG-19	GQA-2KV	-6.1%	-8.8%	-12.3%
PG-19	KVPress-Knorm	-12.4%	-14.7%	-14.0%
PG-19	KVPress-StreamingLLM	-10.6%	-3.0%	+8.7%

Table 2 reports mean throughput changes relative to the baseline mean for the same dataset and prompt length. For example, each 512-context entry uses the first 512 tokens as prompt, decodes 64 tokens, averages three measured runs, and divides by the corresponding baseline throughput. On WikiText, KVPress improves 128/512-token generation, but both KVPress variants slow down at 1024 tokens. StreamingLLM gives +35.3% and +50.3% at 128 and 512 tokens, respectively. On PG-19, StreamingLLM is close to baseline at 512 tokens and gives a modest +8.7% at 1024 tokens. GQA stores fewer KV heads but is not a useful accelerator because its quality collapses and its speed is inconsistent.

The raw TTFT, TPOT, end-to-end time, throughput means, standard deviations, and per-run values are included in `outputs/comparison_speed.*`. The CPU results are mixed: compression helps in some settings but not all, and small deltas should not be over-interpreted. A GPU implementation with fused kernels and larger models would be a more favorable setting for cache compression and reduced-KV attention.

5 Conclusion

The most reliable positive results are KVPress on WikiText at 128/512-token contexts and StreamingLLM on PG-19 at 1024 tokens, where the 3-run mean throughput improves while the cache is explicitly compressed by 50%. Reduced-KV GQA is not successful despite storing only 2 KV heads: it sharply worsens perplexity and does not consistently accelerate generation. Overall, training-free KV cache compression is a practical direction for inference acceleration, but its benefit depends on implementation overhead, dataset/runtime behavior, and model size.

References

- Biderman, S., Schoelkopf, H., Anthony, Q., Bradley, H., O’Brien, K., Hallahan, E., Khan, M. A., Purohit, S., Prashanth, U. S., Raff, E., Skowron, A., Sutawika, L., and Van Der Wal, O. Pythia: A suite for analyzing large language models across training and scaling. *International Conference on Machine Learning*, 2023.
- Ainslie, J., Lee-Thorp, J., de Jong, M., Zemlyanskiy, Y., Lebron, F., and Sanghai, S. GQA: Training generalized multi-query transformer models from multi-head checkpoints. *Empirical Methods in Natural Language Processing*, 2023.
- Dao, T., Fu, D. Y., Ermon, S., Rudra, A., and Re, C. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. *Advances in Neural Information Processing Systems*, 2022.

- 113 Xiao, G., Tian, Y., Chen, B., Han, S., and Lewis, M. Efficient streaming language models with attention sinks.
114 arXiv:2309.17453, 2023.
- 115 NVIDIA. KVPress: KV cache compression for efficient LLM inference. [https://github.com/NVIDIA/](https://github.com/NVIDIA/kvpress)
116 kvpress, 2024.
- 117 Merity, S., Xiong, C., Bradbury, J., and Socher, R. Pointer sentinel mixture models. arXiv:1609.07843, 2016.
- 118 Rae, J. W., Potapenko, A., Jayakumar, S. M., and Lillicrap, T. P. Compressive transformers for long-range
119 sequence modelling. *International Conference on Learning Representations*, 2020.